

Массовая выборка и вставка записей в БД

В классе [ITableCopier](#) реализованы методы для массовой выборки и вставки записей в БД. Класс применяется для драйверов **PostgreSQL** (веб-приложения) и **SQLite** (десктоп- и мобильные приложения). Однако в работе с SQLite есть ряд ограничений по использованию методов. ITableCopier позволяет скопировать большой объем данных одним запросом, а не с помощью генерации запроса для каждой отдельной записи.

Выборка данных

Метод Get

Чтобы получить таблицу целиком или ее часть с сервера БД, вызовите метод [Get](#).

Формат запроса: **Get(self, output, options, filter)**, где:

- self - обращение к текущему экземпляру класса (только для Python),
- output - поток для получения данных,
- options - параметры копирования,
- filter - фильтр получаемых данных (необязательный параметр).

Python:

```
1 copier = sbis.CreateTableCopier(fmt)
2 with io.BytesIO() as text:
3     copier.Get(
4         text,
5         sbis.TableCopyOptions(
6             format=sbis.TableCopyFormat.Text,
7             encoding=sbis.EncodingType.UTF8
8         )
9     )
```

C++:

```
1 std::ostream text;
2
3 auto copier = db->CreateTableCopier( fmt );
4 copier->Get( text, { sbis::dbi::TableCopyFormat::Text,
5                     sbis::Encoding::UTF8 } );
```

Метод GetByQuery

Метод [GetByQuery](#) позволяет получить таблицу или ее часть с помощью SELECT-запроса.

Формат запроса: **GetByQuery(self, query, cb, options)**, где:

- self - обращение к текущему экземпляру класса (только для Python),
- query - SELECT-запрос,
- cb - обратная связь для получения данных,
- options - параметры копирования.

Python:

```
1 class Collector:
2     def __init__(self):
```

```

3         self.data = bytearray()
4     def __call__(self, chunk):
5         self.data += chunk
6 collector = Collector()
7
8 copier = sbis.CreateTableCopier(fmt)
9 copier.GetByQuery(
10     query,
11     collector,
12     sbis.TableCopyOptions(
13         format=sbis.TableCopyFormat.Binary,
14         encoding=sbis.EncodingType.UTF8
15     )
16 )

```

C++:

```

1 std::vector< char > data;
2
3 auto copier = db->CreateTableCopier( fmt );
4 copier->GetByQuery( query,
5     [ &data ]( char* buffer, size_t size ) {
6     data.insert( data.end(), buffer, buffer + size ); },
7     { sbis::dbi::TableCopyFormat::Binary,
8       sbis::Encoding::UTF8 } );

```

Вставка данных

Метод Put

Чтобы открыть сеанс записи, создайте объект [TableCopyIn](#).

Метод [Put](#) отправляет данные на сервер БД. Если нет открытого сеанса записи, то сеанс открывается, данные отправляются, и сеанс закрывается; после этого можно считать, что записи успешно добавлены, если не было исключения. Если же сеанс записи уже открыт, то данные отправляются, но проверка корректности откладывается до завершения сеанса записи.

Формат запроса: **Put(self, input, options)**, где:

- self - первая переменная метода класса,
- input - буфер или поток с данными,
- options - параметры копирования.

Python:

```

1 copier = sbis.CreateTableCopier(fmt)
2 with open(file_name, 'rb') as csv:
3     copier.Put(
4         csv,
5         sbis.TableCopyOptions(
6             format=sbis.TableCopyFormat.CSV,
7             encoding=sbis.EncodingType.UTF8
8         )
9     )

```

C++:

```

1 std::ifstream csv;
2 sbis::filesystem::OpenBinaryFile( csv, file_name );
3

```

```
4 auto copier = db->CreateTableCopier( fmt );
5 copier->Put( csv, { sbis::dbi::TableCopyFormat::CSV,
  sbis::Encoding::UTF8 } );
```

Метод [AddRecord](#) добавляет записи в буфер.

Метод [Flush](#) отправляет накопленные данные в БД.

Перегрузка оператора <<

Для копирования данных и БД используется перегрузка оператора <<.

```
1 auto copier = db->CreateTableCopier( fmt );
2 copier->SetCopyInMode( sbis::dbi::TableCopyInMode::InsertOrReplace );
3 *copier << 42;
4 *copier << L"something";
5 *copier << 3.14;
6 *copier << false;
7 *copier << sbis::DateTime::Now();
8 copier->Flush();
```



Примечание

Драйвером БД SQLite не поддерживаются стандартные методы класса ITableCopier: Get, GetByQuery, Put.